2021 4th International Conference on Information Communication and Signal Processing

Task Offloading for Mixed Cloud/Fog Computing in Vehicular Network Using Genetic Particle Swarm Optimization

Youchao Deng and Ping Wang
College of Electronics and Information Engineering
Tongji University
ShangHai, China
{1930708, pwang}@tongji.edu.cn

Lin Li
Shanghai International Automobile City Group Co.,Ltd
ShangHai, China
lilin@siac-sh.com

2021 4th International Conference on Information Communication and Signal Processing (ICICSP) | 978-1-6654-0757-1/21/$31.00 ©2021 IEEE | DOI: 10.1109/ICICSP54369.2021.9611841

***Abstract*—Intelligent connected vehicle (ICV) is considered to be a key technology to solve traffic safety and congestion problems. With the development of ICV technology, vehicles need to perform complex image processing and control decision with deep learning operations. These vehicular applications require a lot of computing power while computation capability of vehicle is limited. The joint application of cloud computing and fog computing in the vehicular network can provide vehicles with sufficient computing resources through reasonable task offloading scheme, and reduce high communication delay. The paper formulates a problem to minimize the total service latency of all tasks in the system, considering the effect of vehicle movement, communication bandwidth limitations, and task queuing time. A novel hybrid metaheuristic algorithm named improved genetic binary particle swarm optimization (IGBPSO) is proposed to solve this problem, which combines the operations of particle swarm optimization (PSO) and genetic algorithm (GA). Experimental results demonstrate that the algorithm has both fast convergence and strong global search capabilities, which can effectively reduce the total service latency for all tasks and improve the utilization of system resources.**

***Keywords*—task offloading, cloud and fog hybrid computing, vehicular network, binary particle swarm optimization (BPSO), genetic algorithm (GA)**

I. Introduction

In order to solve the increasingly serious problems of traffic safety and traffic congestion, the concept of intelligent connected vehicle (ICV) came into being. The ICV combines the technologies of ADAS (Advanced Driver Assistance Systems) and IoV (Internet of Vehicles). ADAS makes vehicle intelligent by installing a large number of sensors on the vehicle, such as cameras, microwave radar and lidar, while IoV makes vehicle available to exchange information with everything around it by deploying communication equipment on the vehicles and surrounding things. Automobile intelligence and interconnection promotes the surge of in-vehicle applications, but the computation capability of vehicle is limited which seriously affects the user experience [1].

As is well known, cloud computing can provide sufficient computing resources for vehicles. However, task data needs to be transmitted to a remote cloud server through the core network which may lead to intolerable transmission delay. On the contrary, fog computing moves computing resources to the side close to the user which effectively reduces task transmission latency, while its computing resources is relatively limited [2]. Therefore, an appropriate offloading strategy determines whether the on-board task can be completed in time and whether the system resources are effectively utilized.

Myriad researches have been studied on the task offloading in vehicular network. According to the offloading method, researches can be divided into two categories. One is V2I (vehicle to infrastructure) offloading, where vehicles offload tasks to the edge servers on the roadside. In [3], authors proposed an adaptive task offloading scheme to maximize the completion ratio of time-critical tasks in a two-layer vehicular fog computing (VFC) architecture. In [4] and [5], reinforcement learning was also introduced to solve vehicular computation offloading problem. The other is V2V (vehicle to vehicle) offloading, where tasks are offloaded to surrounding vehicles. A problem of minimizing service latency and quality loss of tasks was proposed in the vehicular fog network (VFN), and solved by linear programming optimization (LPO) and binary particle swarm optimization (BPSO) in [6]. Considering the situation of vehicles entering and leaving VFN, authors modeled the offloading problem as a Markov decision process to maximize the long-term benefits of the system in [7]. Meanwhile, there are few researches that combines V2I and V2V offloading, such as [8]. However, reference [8] only considered single task offloading in a proportionate manner while there was a lack of the situation of multiple vehicles and multiple tasks offloading at the same time.

With above motivations, we propose a hybrid architecture which federated cloud computing, fog computing (include V2I and V2V offloading) and local computation. A problem of minimizing total service latency of multiple tasks in multiple vehicles is formulated, considering the impact of vehicle movement in detail. Furthermore, we design an algorithm named improved genetic binary particle swarm optimization (IGBPSO) to solve the problem. IGBPSO combines BPSO with genetic algorithm (GA) enhancing its global search

978-1-6654-0757-1/21/$31.00 ©2021 IEEE

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:21:24 UTC from IEEE Xplore. Restrictions apply.

ability.

The organization of the rest of this paper shows as follows. Section II introduces the system model. Section III presents IGBPSO algorithm. Section IV discusses the experiment parameter and result before we conclude in Section V.

II. System Model and Problem Formulation

Proposed offloading system as shown in Fig. 1, there are a large number of vehicles, lots of road side units (RSUs), multiple edge servers, a number of base stations (BS) and a cloud server center in a two-way highway. Vehicles, RSUs and edge severs constitute the vehicular fog network, while the cloud server represents the cloud computing. Special, RSUs and its corresponding edge servers are regard as static fog nodes (SFNs). According to the communication coverage range L_{V2I} of each RSU, the highway can be divided into several segment. In each segment, the only SFN serves as a control center. All nodes (e.g. vehicles and cloud server) need to periodically upload their own status information (e.g. locations, available resources, etc.) to the control center. Vehicles are also divided into two parts according to the amount of computing resources. One is called client vehicle with scarce resources, and the other part is service vehicle with excess computing resources. Service vehicles are regarded as mobile fog nodes (MFNs). Moreover, SFNs, MFNs and cloud node are uniformly defined as service nodes which can process tasks that are offloaded from client vehicles.

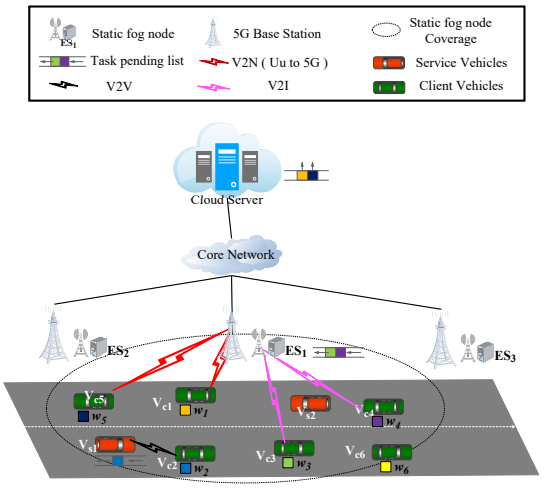


Fig. 1. Cloud and fog computing hybrid architecture in the IoV.

After generating tasks, client vehicle will make a service request containing the task information to control center immediately. Then control center makes a optimal offloading schedule according to global information and notifies the vehicle to offload the task. As show in Fig. 1, V_{c3} and V_{c4} offload tasks to SFN ES_1. Then offloaded tasks are queued in the ES_1's task pending list and will be processed follow a first-come-and-first-served policy. The more powerful computation capability of the service node can reduce processing latency, but inappropriate task schedule may lead to high waiting delay.

A. Definitions

In this subsection, the notations and definition of all system element is detail presented. $M = \{V_{s1}, V_{s2}, \cdots, V_{s|M|}\}$, $\{r\}$, $\{c\}$ denote the set of MFNs, SFN, and cloud node, respectively. Each service node $j \in \{M \cup \{r\} \cup \{c\}\}$ is defined by 2 fundamental attributes $\{f_j, \rho_j\}$, where f_j represents the computation capability, while ρ_j means the number of communication channels. Similarly, $N = \{V_{c1}, V_{c2}, \cdots, V_{c|N|}\}$ is denoted as the set of client vehicles. To simplify the system, we assumed that each client vehicle generates one task. Then the set of tasks is denoted as $W = \{w_1, w_2, \cdots, w_{|N|}\}$. Task w_i is generated by the client vehicle V_{ci}. And it is indivisible and described by 3 properties $\{\theta_i, \alpha_i, \tau_i\}$, where θ_i represents the data size of w_i, α_i stands for the computational complexity, and τ_i represents the deadline for the task processing. Furthermore, A binary variable x_{ij} is indicated whether the task w_i is scheduled to be offload to service node j.

$$x_{ij} = \begin{cases} 1, & w_i \text{ is offloaded to node } j; \\ 0, & \text{Otherwise.} \end{cases} \tag{1}$$

After being assigned to a service node, a task needs to go through 4 procedures including data transmission, queuing, processing, and returning results. So the total service latency for task w_i to be offloaded to node j is expressed as follow.

$$t_{ij}^{\text{task}} = t_{ij}^{\text{trans}} + t_{ij}^{\text{wait}} + t_{ij}^{\text{exe}} + t_{ij}^{\text{result}} \tag{2}$$

where t_{ij}^{trans} represents the time required for data transmission to the node j; t_{ij}^{wait} represents the queuing time in the node j; t_{ij}^{exe} represents the processing time of w_i; and t_{ij}^{result} represents the time required to return the result from the node j to the vehicle V_{ci}. Usually, because of the small amount of result data, t_{ij}^{result} is far less than others, which is negligible.

B. Communication Model

In the proposed heterogeneous network, nodes can communicate with each other. SFNs, BS and cloud server are connected by fiber-wired link, while client vehicle communicates with SFN, service vehicles and BS through V2I communication, V2V communication and vehicle-to-network (V2N) communication, respectively. Different type nodes use different communication interfaces, where V2I and V2V utilize PC5 interface, while V2N uses Uu interface for access to 5G network [9].

1) V2I communication: Transmission channels are frequency-flat block fading Rayleigh channel. $d_{ir}^{-\sigma}$ is the path loss between vehicle V_{ci} and SFN, where σ denotes path loss exponent. Here, h denotes fading coefficient, N_0 denotes the power of white Guassian noise. Then V2I communication rate can represent as follow.

$$R_{V2I}^{ir} = B_{V2I} \log_2(1 + \frac{P_t d_{ir}^{-\sigma} h^2}{N_0}) \tag{3}$$

where d_{ir} denotes the distance between vehicle V_{ci} and RSU, B_{V2I} represents the communication bandwidth of V2I channels, and P_t denotes the transmit power of client vehicle.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:21:24 UTC from IEEE Xplore. Restrictions apply.

In proposed scenario, each vehicle moves at a different constant speed along the lane direction (i.e. x-axis direction), d_{ir} constantly changes because of the high speed movement of vehicle. RSU is located at road segment center where is defined as the coordinate origin. (x, y) denotes as the coordinate of vehicle V_{ci}, v_i is defined as the velocity of V_{ci}. Then, we have

$$d_{ir}(t) = \sqrt{y^2 + (x - vt)^2} \tag{4}$$

It is worth mentioning that vehicle may leave the communication range of service node because of high-speed movement. So denote t_{ij}^{stay} as the time that vehicle V_{ci} stay in the communication range of node j. V2I data rate R_{V2I}^{ir} varies with d_{ir}, thus the average rate during the time V_{ci} keeps connected is

$$\overline{R_{V2I}^{ir}} = \frac{\int_0^{t_{ij}^{\text{stay}}} R_{V2I}^{ir}(t)dt}{t_{ij}^{\text{stay}}} \tag{5}$$

2) V2V communication: The calculation of the communication rate of V2V communication R_{V2V}^{im} between client vehicle V_{ci} and service vehicle V_{sm} is similar with V2I communication. It should be noted that both V_{ci} and V_{sm} are in motion. So we need to use relative coordinate $(x_{relative}, y_{relative})$ and relative speed $v_{relative}$ to calculate the distance d_{im} between different vehicles.

3) V2N communication: In V2N communication, task data is uploaded to BS by 5G new radio (NR) at first, then it need to be sent to cloud server through core network. Data rate is high in NR, while it is limited in the core network considering the congestion. So we defined that R_{V2N} represents the average transmission rate from a vehicle to cloud server.

C. Analytic Model

According to the previous introduction, there are 4 ways to offload and process a task:

1) Local offloading: Considering processing the task locally, there is no need to upload data to another node, so $t_{il}^{\text{trans}} = 0$; and the vehicle only needs to process one task generated by itself without waiting, $t_{il}^{\text{wait}} = 0$. Then, we have

$$t_{il}^{\text{task}} = t_{il}^{\text{exe}} = \frac{\alpha_i \theta_i}{f_{V_{ci}}} \tag{6}$$

2) SFN offloading: Considering offloading the task to the nearest SFN, base on the communication model described above, we have

$$t_{ir}^{\text{trans}} = \frac{\theta_i}{\overline{R_{V2I}^{ir}}}, t_{ir}^{\text{wait}} = \frac{\sum_{p=1}^{|N|} x_{pr}^{\text{before}_i} \alpha_p \theta_p}{f_r}, t_{ir}^{\text{exe}} = \frac{\alpha_i \theta_i}{f_r} \tag{7}$$

$$t_{ir}^{\text{task}} = t_{ir}^{\text{trans}} + t_{ir}^{\text{wait}} + t_{ir}^{\text{exe}} \tag{8}$$

where $x_{pr}^{\text{before}_i}$ represent the tasks arrived and processed before w_i in the SFN.

3) MFN offloading: Considering offloading the task to the MFN V_{sm}, we stipulate that a service vehicle can only receive at most one offloaded task, so $t_{im}^{\text{wait}} = 0$. Then we have

$$t_{im}^{\text{trans}} = \frac{\theta_i}{R_{V2V}^{im}}, t_{im}^{\text{exe}} = \frac{\alpha_i \theta_i}{f_{V_{sm}}} \tag{9}$$

$$t_{im}^{\text{task}} = t_{im}^{\text{trans}} + t_{im}^{\text{exe}} \tag{10}$$

4) Cloud offloading: Considering offloading the task to cloud node. Cloud server is assumed that has infinity computing resources. Once the task is uploaded to the cloud server, it will be completed immediately. So there is no waiting time and processing time. The total task service latency is only related to the time for data transmission.

$$t_{ic}^{\text{task}} = t_{ic}^{\text{trans}} = \frac{\theta_i}{R_{V2N}} \tag{11}$$

D. Problem Formulation

According to the given offloading strategy, we can calculate the service latency of the task w_i as follow.

$$t_i^{\text{task}} = x_{il} \cdot t_{il}^{\text{task}} + x_{ir} \cdot t_{ir}^{\text{task}} + \sum_{m=1}^{|M|} x_{im} \cdot t_{im}^{\text{task}} + x_{ic} \cdot t_{ic}^{\text{task}} \tag{12}$$

In this paper, we minimize the total service latency for all tasks to improve utility of system resource. Therefore, the optimization problem is formulated as follow.

$$\begin{aligned}
\mathbf{P1}: \min_{\forall x_{ij}} \quad & T^{\text{sum}} = \sum_{i=1}^{|N|} t_i^{\text{task}} \\
s.t. \quad & \text{C1:} \quad x_{il}, x_{im}, x_{ir}, x_{ic} \in \{0, 1\} \\
& \text{C2:} \quad x_{il} + \sum_{m=1}^{|M|} x_{im} + x_{ir} + x_{ic} = 1 \\
& \text{C3:} \quad x_{ij} \cdot t_{ij}^{\text{trans}} \leq t_{ij}^{\text{stay}} \\
& \text{C4:} \quad \sum_{i=1}^{|N|} x_{ir} \leq \rho_r \\
& \text{C5:} \quad \sum_{i=1}^{|N|} x_{ic} \leq \rho_c \\
& \text{C6:} \quad \sum_{i=1}^{|N|} x_{im} \leq \rho_m \\
& \text{C7:} \quad t_i^{\text{task}} \leq \tau_i
\end{aligned} \tag{13}$$

Constraints C1 and C2 indicate that each task must be offloaded to at one node, including processing locally. Constraint C3 implies that the data have to be transmitted to a service node before the vehicle leaves its coverage. Constraints C4, C5 and C6 represent that the number of tasks transmitted at the same time cannot exceed the bandwidth capacities of the SFN , cloud node and MFNs, respectively. C7 stands for that each task must be processed before the deadline.

III. Offloading Algorithm with Improved Genetic Particle Swarm Optimization

The problem **P1** is a nonlinear 0-1 programming problem which has long been proved to be an NP-hard problem. There is no algorithm to find the optimal solution of the problem in

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:21:24 UTC from IEEE Xplore. Restrictions apply.

polynomial time. Therefore, we propose a heuristic algorithm to solve this problem. Before applying the heuristic algorithm, we use the penalty function method [10] to convert the complicated constraints of problem **P1** into the corresponding penalty functions, then **P1** becomes unconstrained and the solution space becomes continuous, which effectively reduces the difficulty of solving the problems. After the conversion, the problem changes as follow.

$$\min_{\forall x_{ij}} \quad \widetilde{T} = T^{\text{sum}} + \eta P(x) \tag{14}$$

$\widetilde{T}$ is defined as a new object function and η is large positive constant. $P(x)$ is the total penalty of all constraints when they are not met. It is calculated as follow.

$$P(x) = \sum_{p=1}^{\eta^{\neq}} (max\{0, g_p(\boldsymbol{x})\})^{\beta_1} + \sum_{q=1}^{\eta^{=}} |h_q(\boldsymbol{x})|^{\beta_2}. \tag{15}$$

Write the original problem in the standard form of the optimization problem, then $g_p(\boldsymbol{x})$ represents the inequality constraint in the problem, and $h_q(\boldsymbol{x})$ represents the equality constraint of the original problem. $\eta^{\neq}$ indicates the number of inequality constraints, $\eta^{=}$ indicates the number of equality constraints. β_1 and β_2 are two positive constants. After constraints converting to the penalty functions, it can be seen that the value of the penalty function is greater or equal to 0 (e.g. C3 is $(x_{ij} \cdot t_{ij}^{\text{trans}} - t_{ij}^{stay} \leq 0)$, then it is covert to penalty function $(max\{0, x_{ij} \cdot t_{ij}^{\text{trans}} - t_{ij}^{stay}\})^{\beta_1} \geq 0)$. So the optimal solution of the converted problem is same as the original problem.

Among various heuristic algorithms, PSO is easy to implement, and has a faster convergence speed and pretty effect, which has been widely used in engineering. However, PSO is prone to premature convergence (especially in the processing of complex multi-peak search problems). In order to make up for this shortcoming, we proposed an algorithm named IGBPSO which combines BPSO with GA, enhancing its global search capabilities.

In PSO, potential solution is called particle. In this paper, x^{kd} denotes the dth particle in kth generation. It includes $\mathbb{X} = N \times (M+3)$ elements, which each row represents the offloading decision of a task, and each column represents a service node. For example, $x_{im}^{kd} = 1$ represents the task w_i is offloaded to V_{sm}, $x_{i(M+1)}^{kd} = 1$ represents the task w_i processing locally, $x_{i(M+2)}^{kd} = 1$ represents the task w_i is offloaded to SFN, and $x_{i(M+3)}^{kd} = 1$ represents cloud offloading. And v^{kd} is defined as x^{kd} corresponding update velocity. The position and velocity of each particle are updated based on the optimal position of the particle has experienced X_{best}^d and the global optimal position of all particles G_{best}. The standard BPSO update equation is as follows.

$$v^{kd} = wv^{kd} + c_1 r_1 (X_{\text{best}}^d - x^{kd}) + c_2 r_2 (G_{\text{best}} - x^{kd}) \tag{16}$$

$$v_{ij}^{kd\prime} = sigmoid(v_{ij}^{kd}) = \frac{1}{1 + e^{-v_{ij}^{kd}}} \tag{17}$$

$$x_{ij}^{kd} = \begin{cases} 1, & r_{ij} \leq v_{ij}^{kd\prime} \\ 0, & \text{Otherwise.} \end{cases} \tag{18}$$

Algorithm 1 Improved Genetic Particle Swarm Optimization

Input:
Unassigned task set W and its parameter
Client vehicle set N and its parameter
Service node set $\{M \cup \{r\} \cup \{c\}\}$ and its parameter

Output:
Assignment decision set G_{best}
The minimal total service latency $T_{\text{min}}^{\text{sum}}$

1: Set the mutation probability P_{muta} and the parameter of BPSO which include w_{max}, w_{min}, c_1, and c_2
2: Randomly initialize velocities and positions of particles and met constraints C1-C6
3: Calculate fitness value $\widetilde{T}$ of each particle with (14)
4: Update X_{best}^d and G_{best}
5: Set $k = 1$, and k_{max} represents the maximum number of iterations.
6: **while** $k < k_{\text{max}}$ **do**
7: Calculate velocities of particles in PSO with (19-20)
8: Change position of particles in new generation with equation (21)
9: Calculate fitness value ($\widetilde{T}$) of each particle by using (14), then update X_{best}^d and G_{best}
10: Perform GA's crossover operation on X_{best}^d and G_{best} utilizing the operation of single-point crossover to produce two offspring.
11: Perform GA's mutation operation on each row of two offspring with a certain probability P_{muta} to obtain two mutated particle
12: Calculate fitness value of above each new particle including offspring and mutated particles, select the best one $\hat{x}$
13: **if** $\hat{x}$ better than X_{best}^d and G_{best} **then**
14: Update the locally optimal position (X_{best}^d) of each particle d and the globally optimal position (G_{best}) of the population
15: **end if**
16: $w = (w_{\text{max}} - w_{\text{min}}) \cdot \dfrac{k_{\text{max}} - k}{k_{\text{max}}} + w_{\text{min}}$
17: $k = k + 1$
18: **end while**
19: **return** G_{best} and $T_{\text{min}}^{\text{sum}}$

where w is the dynamically changing inertia weight, as show in the 16th line of Algorithm 1, c_1 and c_2 are acceleration constants, r_1 and r_2 are two random functions with a range $[0, 1]$.

However, the standard BPSO has two disadvantages including difficulty in choosing suitable value for inertia weight w and lack of utilization of the last moment position. To this end, we use a novel BPSO algorithm proposed in [11].

Similar to the definition of velocity in continuous version of PSO, v_{ij}^1 and v_{ij}^0 are introduced, which means the probability of the bits x_{ij} change to 0 or 1, respectively. The update

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:21:24 UTC from IEEE Xplore. Restrictions apply.

equation of them is as follow.

$$\begin{aligned} v_{ij}^0 &= wv_{ij}^0 + d_{ij,1}^0 + d_{ij,2}^0 \\ v_{ij}^1 &= wv_{ij}^1 + d_{ij,1}^1 + d_{ij,2}^1 \end{aligned} \tag{19}$$

where $d_{ij,1}^0$ and $d_{ij,1}^1$ represent the influence of X_{best}^d, while $d_{ij,2}^0$ and $d_{ij,2}^1$ represent the influence of G_{best}.

$$\begin{aligned} &\text{If } X_{\text{best}}^d(i,j) = 1 \quad \text{Then } d_{ij,1}^1 = c_1 r_1 \text{ and } d_{ij,1}^0 = -c_1 r_1 \\ &\text{If } X_{\text{best}}^d(i,j) = 0 \quad \text{Then } d_{ij,1}^0 = c_1 r_1 \text{ and } d_{ij,1}^1 = -c_1 r_1 \\ &\text{If } G_{\text{best}}(i,j) = 1 \quad \text{Then } d_{ij,2}^1 = c_2 r_2 \text{ and } d_{ij,2}^0 = -c_2 r_2 \\ &\text{If } G_{\text{best}}(i,j) = 0 \quad \text{Then } d_{ij,2}^0 = c_2 r_2 \text{ and } d_{ij,2}^1 = -c_2 r_2 \end{aligned}$$

Then, choose the proper velocity as the final according to the value of x_{ij}^{kd}.

$$v_{ij}^{kd} = \begin{cases} v_{ij}^1 & , \text{ if } x_{ij}^{kd} = 0 \\ v_{ij}^0 & , \text{ if } x_{ij}^{kd} = 1 \end{cases} \tag{20}$$

And the next state of particle x_{ij}^{kd} is calculated as follow.

$$x_{ij}^{(k+1)d} = \begin{cases} \bar{x}_{ij}^{kd} & \text{if } r_{ij} \leq sigmoid(v_{ij}^{kd}), \\ x_{ij}^{kd} & \text{otherwise.} \end{cases} \tag{21}$$

where $\bar{x}_{ij}^{kd}$ is the 2's complement of x_{ij}^{kd} and r_{ij} is a random number between 0 and 1.

After obtaining the next generation of particles according to the above steps, X_{best}^d and G_{best} need to update. Calculating the fitness of each new particle by (14), if the fitness of new particle x^{kd} is less than the fitness of X_{best}^d, then $X_{\text{best}}^d = x^{kd}$. The same is true for the global optimal position G_{best}. Then, we conduct a single-point-crossover operation for X_{best}^d and G_{best}. For the offspring generated after the crossover operation, mutation operation is conducted on each row with a specific mutation probability P_{muta} [10]. When a gene mutation occurs in row i, task w_i randomly selects a new service node j' (i.e. sets $x_{ij'} = 1$). Furthermore, we perform natural selection operations based on the fitness of these newly generated particles, keeping the particle with the smallest fitness as new X_{best}^d and G_{best}. The above genetic manipulations can prevent premature convergence. The overall process of the IGBPSO algorithm is as shown in Algorithm 1.

IV. Simulation Results and Discussions

A. Parameter Setting

According to [7] and [8], parameters of communication model are given as follows. $B_{V2I} = B_{V2V} = 10$ MHz; $h = 0.98$, $N0 = 1.74 \times 10^{-11}$, $P_f = 23$ dBm, $\delta = 4$, $\rho_r = 7$, $\rho_c = 10$, $\rho_m = 1$; R_{V2N} is set in the range of $[2, 3]$ Mbps, the communication radius of V2I and V2V is 300m and 200m, respectively. Computing capability of each node is defined as follow. $f_{V_{ci}} \in [1, 4] * 10^9$ cycles/s, $f_{V_{sm}} \in [3, 6] * 10^9$ cycles/s, $f_r = 9 * 10^9$ cycles/s. Parameters of each task are given as follow. $\alpha_i \in [100, 5000]$, $\theta_i \in [1 * 10^5, 5 * 10^6]$ bits.

It is worth noting that the setting of parameters plays an important role in the effect of typical meta-heuristic optimization algorithms. Parameters of IGBPSO are given as follows: $w_{\max} = 0.8, w_{\min} = 0.4$, $c_1 = 2, c_2 = 2$, $P_{\text{muta}} = 0.01$, $k_{\max} = 200$, the number of particles D is 100.

B. Experimental Results

In order to show the effectiveness of the algorithm proposed in this paper, we compared IGBPSO with four other offloading strategies, as shown below.

- All Local Offloading (ALO) : All tasks are processed locally.
- All Cloud Offloading (ACO) : All tasks are offloaded to cloud server. Due to the limitation of the number of communication channels, all tasks cannot be offloaded to one cloud server in the same time. This offloading strategy is only exists in ideals.
- BPSO : The standard BPSO algorithm has fast convergence. The comparison between BPSO and IGBPSO can prove the convergence speed of IGBPSO and determinate whether the addition of gene manipulation brings performance improvement.
- GA : GA has a good global search performance. A comparison with it can highlight whether the results of IGBPSO are effective.

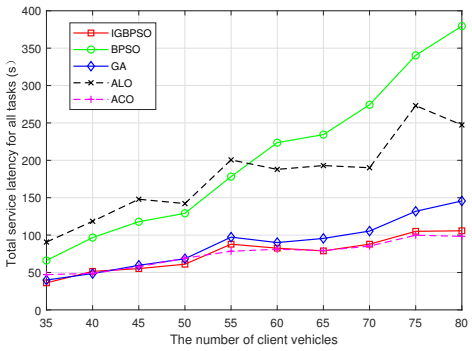


Fig. 2. T^{sum} with respect to different number of client vehicles.

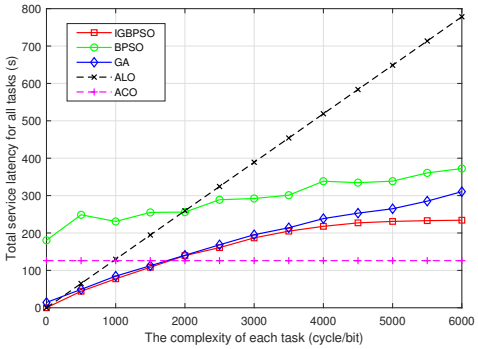


Fig. 3. T^{sum} with respect to different complexity of each task.

Fig. 2 shows the total service latency of each algorithm with varying the number of client vehicles. In this experiment, the complexity and data size of each task, position of each client vehicles fluctuate randomly within a certain range, so as the number of client vehicle increases, the total service latency did not increase linearly. As the number of client vehicles

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:21:24 UTC from IEEE Xplore. Restrictions apply.

increases, algorithm search space expanded, it can be seen that BPSO performs poorly because of falling into a local optimal solution prematurely. Conversely, IGBPSO always chooses optimal service node for each task obtaining lower service latency.

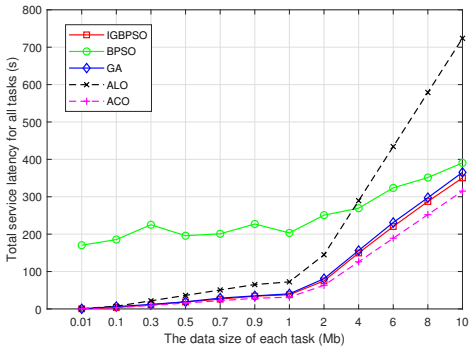


Fig. 4. T^{sum} with respect to different data size of each task.

Fig. 3 demonstrates the total service latency of each algorithm with varying the complexity of task. It can be observed from Fig. 3 that as the complexity increases, the results of all heuristic algorithms are closer to the ACO. The reason is that the increase in computational complexity of each task leads to task requiring more computing resources and more computing latency. With increase of computing latency, the system is more inclined to offload tasks to SFN and cloud node which have more powerful computation capabilities.

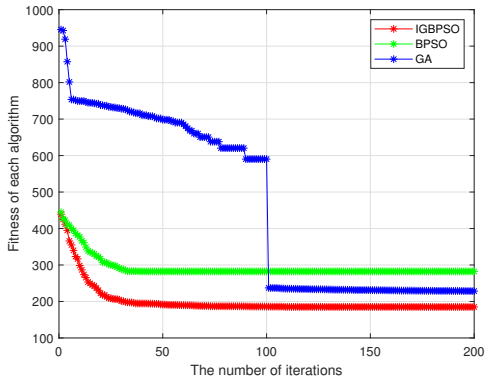


Fig. 5. Fitness of each algorithm over different number of iterations.

Fig. 4 shows the total service latency with respect to the size of task data. Transmission latency, calculation latency and waiting time all increase with the increase of the size of task data. As the size of data increases, IGBPSO and GA achieved similar results and were significantly better than ALO and BPSO. However, as shown in Fig. 5, IGBPSO converged at about the 30th iteration, while GA did not converge until the 100th iteration, which shows that IGBPSO converges faster than GA. Meanwhile, the final result of IGBPSO is also better than the other two algorithms. According to the above experimental results, it can be seen that IGBPSO has both good global search capabilities and extremely fast convergence speed, which can make a reasonable offloading scheme to obtain a lower service latency.

V. Conclusion

This paper proposes a cloud and fog computing hybrid architecture, which is composed of cloud nodes, fog nodes (SFNs and MFNs) and client vehicles. Furthermore, in order to make better use of these heterogeneous service nodes, this paper formulates a task offloading problem to minimize the total service latency for all task. To solve this problem, we designed the IGBPSO algorithm, a heuristic algorithm that integrates GA and BPSO. The simulation results demonstrate that IGBPSO has fast convergence speed and great global search ability. The proposed offloading strategy effectively reduces service latency and improve system resource utilization under this scenarios.

Acknowledgment

This work was supported in part by the National Key R&D Program of China (Grant No. 2018YFB0105101), in part by the Key-Area Research and Development Program of Guangdong Province with grants No. 2019B090912002, and in part by the Science and Technology Commission of Shanghai Municipality (20QB1402300). (Corresponding author:Lin Li.)

References

[1] O. Kaiwartya, A. H. Abdullah, Y. Cao, A. Altameem, M. Prasad, C.-T. Lin, and X. Liu, "Internet of vehicles: Motivation, layered architecture, network model, challenges, and future aspects," *IEEE Access*, vol. 4, pp. 5356–5373, 2016.

[2] Y. Xiao and C. Zhu, "Vehicular fog computing: Vision and challenges," in *2017 IEEE International Conference on Pervasive Computing and Communications Workshops (PerCom Workshops)*. IEEE, 2017, pp. 6–9.

[3] C. Liu, K. Liu, S. Guo, R. Xie, V. C. Lee, and S. H. Son, "Adaptive offloading for time-critical tasks in heterogeneous internet of vehicles," *IEEE Internet of Things Journal*, vol. 7, no. 9, pp. 7999–8011, 2020.

[4] Q. Qi, J. Wang, Z. Ma, H. Sun, Y. Cao, L. Zhang, and J. Liao, "Knowledge-driven service offloading decision for vehicular edge computing: A deep reinforcement learning approach," *IEEE Transactions on Vehicular Technology*, vol. 68, no. 5, pp. 4192–4203, 2019.

[5] J. Wang, T. Lv, P. Huang, and P. T. Mathiopoulos, "Mobility-aware partial computation offloading in vehicular networks: A deep reinforcement learning based scheme," *China Communications*, vol. 17, no. 10, pp. 31–49, 2020.

[6] C. Zhu, J. Tao, G. Pastor, Y. Xiao, Y. Ji, Q. Zhou, Y. Li, and A. Ylä-Jääski, "Folo: Latency and quality optimized task allocation in vehicular fog computing," *IEEE Internet of Things Journal*, vol. 6, no. 3, pp. 4150–4161, 2018.

[7] Q. Wu, H. Ge, H. Liu, Q. Fan, Z. Li, and Z. Wang, "A task offloading scheme in vehicular fog and cloud computing system," *IEEE Access*, vol. 8, pp. 1173–1184, 2019.

[8] H. Wang, X. Li, H. Ji, and H. Zhang, "Federated offloading scheme to minimize latency in mec-enabled vehicular networks," in *2018 IEEE Globecom Workshops (GC Wkshps)*. IEEE, 2018, pp. 1–6.

[9] H. Zhou, W. Xu, J. Chen, and W. Wang, "Evolutionary v2x technologies toward the internet of vehicles: Challenges and opportunities," *Proceedings of the IEEE*, vol. 108, no. 2, pp. 308–323, 2020.

[10] J. Bi, H. Yuan, S. Duanmu, M. C. Zhou, and A. Abusorrah, "Energy-optimized partial computation offloading in mobile edge computing with genetic simulated-annealing-based particle swarm optimization," *IEEE Internet of Things Journal*, 2020.

[11] M. A. Khanesar, M. Teshnehlab, and M. A. Shoorehdeli, "A novel binary particle swarm optimization," in *2007 Mediterranean Conference on Control Automation*, 2007, pp. 1–6.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:21:24 UTC from IEEE Xplore. Restrictions apply.